

n8n-io/n8n

n8n, automatisation de workflows et agents IA : une CI complète et bloquante, mais aucun bug étiqueté ni couverture publiée

Fair-code workflow automation platform with native AI capabilities. Combine visual building with custom code, self-host or cloud, 400+ integrations.

10

FICHIERS DE TEST

Vitest (unit, integration, migration) et Playwright pour l'e2e

1

WORKFLOWS CI

375

ISSUES OUVERTES

<https://github.com/n8n-io/n8n> Licence Sustainable Use License Onboard · profil qa

☑ Que fait ce projet et quels sont ses parcours critiques ?

VERDICT

Trois familles de parcours concentrent les bugs discutés : déclencheurs, moteur d'exécution, agents IA et MCP — commencez vos cas de test là.

n8n est une plateforme d'automatisation de workflows avec des capacités IA natives : canevas visuel, code sur mesure, auto-hébergement ou cloud, plus de 1 500 intégrations annoncées. Le dépôt est actif et public : 203 921 étoiles, 60 629 copies, branche par défaut `master` .

Les parcours à couvrir en priorité ne se devinent pas : ce sont ceux qui cassent déjà. Les dix issues les plus discutées les désignent une par une.

PARCOURS	CE QUI CASSE	ISSUE
Déclencheur Chat	URL de chat publique en « Connection rejected » après montée de version	28891, 25 commentaires
Déclencheur Webhook	déclenchement multiple sur un seul appel	31837, 17 commentaires
Exécution d'un workflow	objet imbriqué muté entre branches sœurs	29127, 15 commentaires
Agents IA et MCP	sous-agents incapables d'utiliser les outils MCP	26373, 15 commentaires
Sous-workflows	données du déclencheur renvoyées au lieu du nœud final	33539, 14 commentaires
Mode queue	appel d'outil MCP en échec côté worker	26394, 12 commentaires

Ces six lignes viennent des issues chaudes relevées¹². Montée de version, exécution et IA reviennent dans presque toutes.

☑ Comment est-il testé aujourd'hui ?

VERDICT

Quatre étages de tests coexistent — unitaire, intégration sur base réelle, migrations, end-to-end Playwright — mais aucun chiffre de couverture n'est publié dans le dépôt.

- ☐ `.agents/` règles de revue pour agents IA
- ☐ `.claude/` config Claude Code
- ☐ `.devcontainer/` environnement de dev conteneurisé
- ☐ `.githubhooks/` hooks git
- ☐ `.github/` workflows CI, actions, scripts
- ☐ `.opencode/` config agent OpenCode
- ☐ `.vscode/` réglages VS Code
- ☐ `assets/` images et ressources
- ☐ `docker/` images Docker
- ☐ `docs/` documentation et schémas générés
- ☐ `packages/` code source, monorepo pnpm
- ☐ `patches/` patches de dépendances pnpm
- ☐ `scripts/` scripts de build et de dev
- ☐ `security/` outillage sécurité
- ☐ `.actrc` config act, Actions en local
- ☐ `.aikido` config scanner sécurité Aikido
- ☐ ... et 37 autres entrées à la racine du dépôt

Il n'y a pas de dossier de tests à la racine : n8n est un monorepo pnpm et chaque package porte les siens. Le repère principal est `packages/cli/test`, 10 entrées à sa racine — 4 dossiers (`unit`, `integration`, `migration`, `shared`) et 6 fichiers d'amorçage.

L'outillage se lit dans les configurations, pas dans un README de test : Vitest pour l'unitaire, l'intégration et les migrations, Playwright pour l'end-to-end. `packages/cli` porte six fichiers `vitest.config*.ts`, dont deux variantes `testcontainers` qui lancent une vraie base en conteneur ; `vitest.workspace.ts` à la racine agrège l'ensemble, et `@stryker-mutator/vitest-runner` en dépendance de développement signale des tests de mutation. L'e2e vit à part, dans `packages/testing/playwright`, aux côtés de `performance/`, `containers/`, `code-health/` et `test-impact/`.

Une seule commande pour tout lancer : `pnpm test`, qui délègue à Turborepo.

Deux chiffres manquent, et ils comptent : le nombre total de fichiers de test du dépôt n'a pas été compté (monorepo, tests répartis par package) et le taux de couverture n'est pas publié dans le dépôt — seul `codecov.yml` atteste d'un suivi. À vérifier sur <https://app.codecov.io/gh/n8n-io/n8n> et <https://github.com/n8n-io/n8n/tree/master/packages>.

☑ Que dit la CI et quand tourne-t-elle ?

VERDICT

La CI est riche et bloquante à la fusion, mais sélective par fichiers modifiés : un « tout vert » sur une pull request ne veut pas dire que l'end-to-end a tourné.

Le pipeline de référence est `CI: Pull Requests (Build, Test, Lint)`, dans `.github/workflows/ci-pull-requests.yml`, déclenché sur `pull_request` et sur `merge_group`. La seconde condition est ce qui protège vraiment `master` : la file de fusion rejoue les contrôles sur la fusion réelle, et le job `required-checks` bloque l'intégration tant que tout n'est pas vert.

Ce qu'il exécute, par étage :

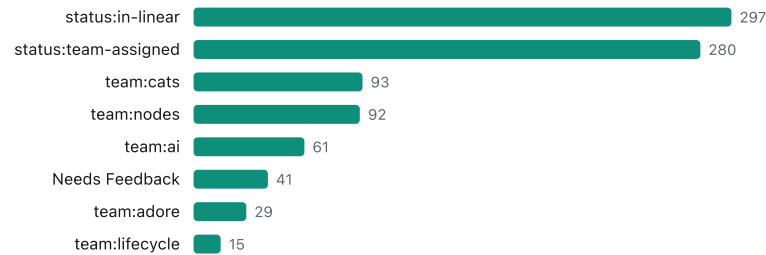
ÉTAGE	JOB OU WORKFLOW APPELÉ	PORTÉE
Build	<code>install-and-build</code>	<code>pnpm build</code> , <code>format:check</code>
Statique	<code>typecheck</code> , <code>lint</code>	<code>turbo typecheck</code> , <code>test-linting-reusable.yml</code>
Unitaire	<code>unit-test</code>	<code>test-unit-reusable.yml</code>
Base de données	<code>db-tests</code>	<code>test-db-reusable.yml</code> , SQLite et Postgres
End-to-end	<code>e2e</code> , <code>e2e-performance</code>	<code>test-e2e-reusable.yml</code> (Playwright), <code>test-e2e-performance-reusable.yml</code>
Livraison	<code>check-packaging</code> , <code>prepare-docker</code> , <code>dev-server-smoke</code>	<code>pnpm -r pack --dry-run</code> , image Docker, démarrage
Sécurité	<code>security-checks</code>	<code>sec-ci-reusable.yml</code>

Le point de vigilance est en amont de ce tableau : une étape `ci-filter` calcule, à partir des fichiers modifiés, quels étages tournent (`unit`, `e2e`, `db`, `db-migrations`, `e2e-performance` ...). Une pull request peut donc être verte sans avoir rejoué l'end-to-end. Les campagnes complètes sont reportées la nuit, dans des workflows dédiés : `test-workflows-nightly.yml`, `test-e2e-coverage-nightly.yml`, `test-benchmark-nightly.yml`, `test-sbom-nightly.yml`.

📁 Où sont les bugs connus ?

VERDICT

Ici, un bug ne porte pas d'étiquette « bug » : filtrez sur `Needs Feedback` et `triage:needs-info` pour trouver ce qui attend une reproduction.



375 issues ouvertes, 300 fermées sur les trente derniers jours : le flux est traité, pas accumulé. Le compteur global du dépôt affiche 1 141 éléments ouverts, pull requests comprises.

Le piège est dans les étiquettes. Sur les 15 labels relevés, aucun ne s'appelle `bug` : ce sont des labels d'équipe — `team:cats` 93, `team:nodes` 92, `team:ai` 61, `team:adore` 29, `team:lifecycle` 15 — et de statut — `status:in-linear` 297, `status:team-assigned` 280.

Autrement dit, le tri par gravité se fait dans l'outil de tickets interne, pas sur GitHub : on ne peut pas filtrer les bugs par label ici, il faut lire les titres.

Deux étiquettes sont directement exploitables par un testeur : `Needs Feedback`, 41 issues, et `triage:needs-info`, 4 — des tickets bloqués faute de reproduction. Ce sont les meilleures cibles pour un premier apport.

Aucune issue « good first issue » n'a été relevée : il n'y a pas de porte d'entrée balisée. Les dix tickets les plus discutés sont listés plus haut ; les quatre suivants complètent la carte : 25852 chat UI, 32278 enregistrement d'un serveur MCP local, 24986 compteurs d'exécution de la page Overview, 25276 schéma invalide du Vector Store Question Answer Tool³.

☑ Comment reproduire et signaler un bug ici ?

VERDICT

Reproduisez dans le mode de déploiement du plaignant — SQLite, Postgres, queue ou multi-main — avant d'écrire le ticket : c'est là que se joue la moitié des bugs discutés.

Le guide de contribution encadre la marche à suivre : il a une section « Test suite » (unitaires, couverture, end-to-end), des « Community PR Guidelines » et impose un accord de licence contributeur avant toute fusion.

Reproduire, dans l'ordre :

- 1 Sur une version publiée, sans compiler : `docker run -it --rm --name n8n -p 5678:5678 -v n8n_data:/home/node/.n8n docker.n8n.io/n8nio/n8n`, éditeur sur <http://localhost:5678>.
- 2 Sur le code : `pnpm install`, puis `pnpm build`, puis `pnpm start`.
- 3 Sur une base propre, sans casser votre instance : `N8N_USER_FOLDER=~/.n8n3/ pnpm run dev` depuis `packages/cli`.
- 4 Dans la bonne configuration : les piles Testcontainers `stack:sqlite`, `stack:postgres`, `stack:queue`, `stack:multi-main` reproduisent les modes de déploiement. Plusieurs bugs relevés ne se voient qu'en mode queue, par exemple l'issue 26394.

Avant d'ouvrir un ticket, vérifiez qu'un correctif n'est pas déjà en vol : 756 pull requests sont ouvertes, et 167 fusions ont été relevées entre le 7 et le 10 septembre 2026. Vérifiez aussi la branche : les correctifs sont reportés sur `release-candidate/2.38.x`, `release-candidate/2.39.x` et `1.x`, donc un bug corrigé sur `master` ne l'est pas forcément sur votre version. Enfin, les questions d'usage vont au forum communautaire, pas au dépôt.

☑ Quelles zones sont peu couvertes ?

VERDICT

Le débit est le risque principal : trois lignes de version publiées en trois jours, et pas un chiffre de couverture disponible pour savoir ce que la CI laisse passer.

Le rythme est le premier facteur de risque. 251 puis 249 commits sur les deux semaines complètes relevées, dernier commit le 10 septembre 2026 à 13 h 00 UTC, dix versions publiées entre le 7 et le 10 septembre — dont `n8n@2.39.2`, `n8n@2.38.6` et `n8n@1.123.79` le même jour. Trois lignes de version vivent en parallèle : toute campagne de test doit dire sur laquelle elle porte.

La charge est répartie : 166 commits pour les dix premiers contributeurs et un bus factor de 16, soit seize personnes pour la moitié des commits récents. Le risque n'est donc pas la dépendance à une personne, c'est le débit.

Les zones sans mesure, à traiter comme non couvertes jusqu'à preuve du contraire :

- ❑ **Couverture réelle** : aucun taux dans le dépôt, seul `codecov.yml` atteste du suivi → <https://app.codecov.io/gh/n8n-io/n8n>.
- ❑ **Volume de tests par package** : non compté hors `packages/cli/test` → <https://github.com/n8n-io/n8n/tree/master/packages>.
- ❑ **Intégrations** : plus de 1 500 annoncées au README, `team:nodes` porte 92 issues ouvertes — aucune mesure de couverture par nœud.
- ❑ **IA et MCP** : `team:ai` porte 61 issues, et quatre des dix tickets les plus discutés touchent agents, MCP ou vector store.

Les dix semaines antérieures apparaissent à zéro commit dans le relevé : c'est la limite de la fenêtre de mesure, pas un arrêt du projet — le dépôt a reçu du code le jour même.



Plan de test en 5 points

- 1 **Fixer la cible.** Une campagne = une ligne de version et un mode de déploiement. Trois lignes vivent en parallèle (`n8n@2.39.2` , `n8n@2.38.6` , `n8n@1.123.79`) et quatre modes sont outillés (SQLite, Postgres, queue, multi-main).
- 2 **Couvrir d'abord les trois familles qui cassent** : déclencheurs (webhook, chat), moteur d'exécution et sous-workflows, agents IA et MCP. Rejouez les scénarios des issues 28891, 31837, 29127, 26373, 33539 et 26394 comme cas de non-régression.
- 3 **Ne pas se fier au vert d'une pull request.** L'étape `ci-filter` sélectionne les étages selon les fichiers modifiés : exigez un passage end-to-end complet avant livraison, ou appuyez-vous sur les campagnes de nuit `test-workflows-nightly.yml` et `test-e2e-coverage-nightly.yml` .
- 4 **Traiter la couverture comme inconnue.** Aucun taux n'est publié dans le dépôt ; relevez-le sur Codecov avant de décider où écrire des tests, et priorisez les nœuds et l'IA, les deux zones les plus chargées en issues.
- 5 **Rendre vos reproductions utiles au projet.** 41 issues `Needs Feedback` et 4 `triage:needs-info` attendent une reproduction ; le guide de contribution et l'accord de licence contributeur encadrent le dépôt d'un correctif ou d'un cas de test.

RÉFÉRENCES PUBLIQUES

☒ Sources

- 1

<https://github.com/n8n-io/n8n/issues/28891>
- 2

<https://github.com/n8n-io/n8n/issues/31837>
- 3

<https://github.com/n8n-io/n8n/issues/24986>